

Name: Keerthana R

Reg No: 411625104031

Department: B.E CSE

Subject: Java (OOPs)

Batch: 03

[Title: Fruit Information System using Java]

Aim

To design and implement an Object-Oriented Fruit Information System using Java. The program demonstrates core OOP concepts such as abstraction, inheritance, and polymorphism. It allows users to input a fruit name and displays its taste interactively.

Procedure

Step 1: Defining the Abstract Fruit Class

- A Fruit abstract class was created with a protected data member name.
- A constructor was used to initialize the fruit name.
- An abstract method getTaste() was declared to enforce implementation in subclasses.

Step 2: Creating Subclasses (Apple, Banana, Mango)

- Each subclass (Apple, Banana, Mango) extends the Fruit class.
- The getTaste() method was overridden to return the specific taste of each fruit.

Step 3: Implementing the Main Class (Fruitinfo)

- The Fruitinfo class contains the main() method.
- It uses the Scanner class to accept user input for fruit name.
- A switch statement was used to create the appropriate fruit object based on user input.

Step 4: Applying Object-Oriented Concepts

- **Abstraction:** Achieved by declaring getTaste() as an abstract method in the Fruit class.
- **Inheritance:** Subclasses (Apple, Banana, Mango) inherit from the Fruit class.
- **Polymorphism:** The getTaste() method is overridden in each subclass to provide specific behavior.

- **Encapsulation:** Protected data member name ensures controlled access.

Step 5: Executing the Main Class

- The program execution starts in the main() method.
- User inputs a fruit name, and the system displays its taste.

Step 6: Testing The program was tested by:

- Entering valid fruit names (Apple, Banana, Mango).
- Displaying the correct taste for each fruit.
- Handling invalid inputs gracefully.

CODE

```
import java.util.Scanner;

abstract class Fruit{
    protected String name;

    public Fruit(String name){
        this.name = name;
    }

    public abstract String getTaste();
}

class Apple extends Fruit{
    public Apple(String name){
        super(name);
    }

    public String getTaste(){
        return "Sweet";
    }
}

class Banana extends Fruit{
    public Banana(String name){
        super(name);
```

```

}

public String getTaste(){
return "Sweet";
}
}

class Mango extends Fruit{
public Mango(String name){
super(name);
}

public String getTaste(){
return "Juicy";
}
}

public class Fruitinfo{
public static void main(String[] args){
Scanner sc = new Scanner(System.in);
System.out.println("Enter the fruit(Apple,Banana, or Mango):");
String fruitName = sc.nextLine();
Fruit fruitObj = null;
switch(fruitName){
case "Apple":
fruitObj = new Apple("Apple");
break;
case "Banana":
fruitObj = new Banana("Banana");
break;
case "Mango":
fruitObj = new Mango("Mango");
break;

```

default:

```
System.out.println("Invalid Fruit.");
```

```
return;
```

```
}
```

```
System.out.println("Taste: "+fruitObj.getTaste());
```

```
}
```

```
}
```

The screenshot shows the Online Java Compiler IDE interface. The code editor on the left contains the following Java code:

```
1 import java.util.Scanner;
2 abstract class Fruit{
3     protected String name;
4
5     public Fruit(String name){
6         this.name = name;
7     }
8     public abstract String getTaste();
9 }
10 class Apple extends Fruit{
11     public Apple(String name){
12         super(name);
13     }
14     public String getTaste(){
15         return "Sweet";
16     }
17 }
18 class Banana extends Fruit{
19     public Banana(String name){
20         super(name);
21     }
22     public String getTaste(){
23         return "Sweet";
24     }
25 }
26 class Mango extends Fruit{
27     public Mango(String name){
28         super(name);
29     }
30     public String getTaste(){
31         return "Juicy";
32     }
33 }
```

The right panel shows the execution output. The language version is set to JDK 21.0.0, and Interactive Mode is checked. The input arguments field is empty. The output shows the prompt "Enter the fruit(Apple,Banana, or Mango):" followed by the user input "Mango" and the resulting output "Taste: Juicy". The execution time is 8.292 sec(s).

The screenshot shows the Online Java Compiler IDE interface with the complete code. The code editor on the left contains the following Java code:

```
24 }
25 }
26 class Mango extends Fruit{
27     public Mango(String name){
28         super(name);
29     }
30     public String getTaste(){
31         return "Juicy";
32     }
33 }
34 public class FruitInfo{
35     public static void main(String[] args){
36         Scanner sc = new Scanner(System.in);
37         System.out.println("Enter the fruit(Apple,Banana, or Mango):");
38         String fruitName = sc.nextLine();
39         Fruit fruitObj = null;
40         switch(fruitName){
41             case "Apple":
42                 fruitObj = new Apple("Apple");
43                 break;
44             case "Banana":
45                 fruitObj = new Banana("Banana");
46                 break;
47             case "Mango":
48                 fruitObj = new Mango("Mango");
49                 break;
50             default:
51                 System.out.println("Invalid Fruit.");
52                 return;
53         }
54         System.out.println("Taste: "+fruitObj.getTaste());
55     }
56 }
```

The right panel shows the execution output. The language version is set to JDK 21.0.0, and Interactive Mode is checked. The input arguments field is empty. The output shows the prompt "Enter the fruit(Apple,Banana, or Mango):" followed by the user input "Banana" and the resulting output "Taste: Sweet". The execution time is 7.78 sec(s).

Result

The Fruit Information System was successfully implemented using Object-Oriented Programming in Java. The program allows users to input a fruit name and displays its taste accurately. All OOP principles such as abstraction, inheritance, and polymorphism were effectively demonstrated. The system produced correct outputs and handled user input efficiently.